

Local AI Workflows with n8n

Complete Hands-On Lab Manual

n8n · Ollama · Docker · Qdrant · PostgreSQL
Ganpat University

7 Exercises

CPU Only

No API Keys

Free & Open Source

Exercise Overview

#	Exercise	What You Build	Model	Time
1	 AI Code Reviewer	AI code reviewer using webhook + Ollama	tinyllama	15 min
2	 Explain Like I'm 5	ELI5 explainer using prompt engineering	phi3:mini	15 min
3	 AI File Summariser	Document summariser with Docker file handling	phi3:mini	15 min
4	 Simple HTTP Request	Live data fetcher using public REST API	Public API	5 min
5	 AI Calculator	AI Agent with Calculator Tool and memory	qwen2.5:3b	10 min
6	 AI Interaction Logger	AI logger saving interactions to Data Table	qwen2.5:3b	15 min
7	 RAG Pipeline	RAG chatbot — chat with your own PDF	mxbai + qwen2.5:3b	20 min

Part 1 — Introduction & Setup

What is Docker?

Docker packages software into containers — isolated environments that run consistently on any machine. The n8n AI Starter Kit uses Docker to run n8n, Ollama, PostgreSQL, and Qdrant all together with one command.

What is n8n?

n8n is a free, open-source workflow automation platform. You build workflows visually by connecting nodes — each node performs one task. No code required for most workflows.

What is Ollama?

Ollama runs large language models (LLMs) locally on your CPU. No internet, no API keys, no cost — fully private. All AI processing happens on your own machine.

What is Qdrant?

Qdrant is a vector database that stores and searches data based on meaning. It powers the RAG pipeline in Exercise 7, allowing AI to answer questions from your own documents.

Common Setup Commands

Start the Stack

```
# Clone the starter kit (first time only)
git clone https://github.com/n8n-io/self-hosted-ai-starter-kit.git
cd self-hosted-ai-starter-kit

# Start all services in CPU mode
docker compose --profile cpu up -d

# Check all containers are running
docker ps
```

Verify services are running:

n8n dashboard: <http://localhost:5678>

Qdrant dashboard: <http://localhost:6333/dashboard>

Warm-Up Commands — Pull All AI Models

Run these once before the workshop. Models only need to be downloaded once.

```
# Exercise 1 — AI Code Reviewer
docker exec -it ollama ollama pull tinyllama

# Exercise 2 — Explain Like Im 5
docker exec -it ollama ollama pull phi3:mini

# Exercise 3 — AI File Summariser (also uses phi3:mini — already pulled)

# Exercises 4 — Simple HTTP Request (no model needed)

# Exercises 5 & 6 — AI Calculator and Interaction Logger
docker exec -it ollama ollama pull qwen2.5:3b

# Exercise 7 — RAG Pipeline (embedding model)
docker exec -it ollama ollama pull mxbai-embed-large

# Verify all models downloaded
docker exec -it ollama ollama list
```

File Folder Commands — Exercises 3

Exercises 3 reads and writes files inside the n8n Docker container. Run this at the start of each session:

```
# Create the shared folder inside n8n container
docker exec -it n8n mkdir -p /home/node/.n8n-files

# Upload a file from your current folder to n8n
docker cp document.txt n8n:/home/node/.n8n-files/document.txt

# Download a file from n8n to your current folder
docker cp n8n:/home/node/.n8n-files/summary.txt .

# List files inside n8n container
docker exec -it n8n ls /home/node/.n8n-files/
```

⚠ The .n8n-files folder is lost every time Docker restarts. Always run the mkdir command at the start of each session before doing Exercise 3.

Ollama Credential in n8n

When adding any Ollama node in n8n, always use this Base URL:

`http://ollama:11434`

💡 Inside Docker, containers communicate using service names — not localhost. Always use `http://ollama:11434` — never `http://localhost:11434`



Exercise 1 — AI Code Reviewer



15 min



tinyllama



3 Nodes



Ex 1

What You Will Build

A 3-node n8n workflow that accepts any code snippet via HTTP POST, sends it to a local AI model (tinyllama via Ollama), and returns a professional code review identifying issues and improvements. Students submit code through a browser page — no terminal required.

Workflow architecture:

```
Webhook → Basic LLM Chain → Respond to Webhook
          ↑
          Ollama Model (tinyllama)
```

Concept	What it means
Trigger	An HTTP POST request starts the workflow (the Webhook node)
Process	AI transforms the input into something useful (Basic LLM Chain + Ollama)
Respond	The result is sent back to the caller (Respond to Webhook)

Pre-Requisites

- ☐ Docker Desktop is running
- ☐ n8n is accessible at `http://localhost:5678`
- ☐ Ollama container is running inside Docker
- ☐ tinyllama model is pulled (see Step 0 below)

Step 0 — Pull the AI Model

```
docker exec -it ollama ollama pull tinyllama
```

This downloads the tinyllama model (~600 MB) inside the Ollama Docker container. Only needs to be done once. Wait until you see "success" before continuing.

Step 1 — Create a New Workflow in n8n

http://localhost:5678

Step 1a Click + New Workflow — top left of the n8n dashboard

Step 1b Name the workflow — "AI Code Reviewer"

Step 1c Click Save

Step 2 — Add the Webhook Node

Click the + button on the canvas → search for Webhook → select it.

Set these parameters:

HTTP Method: **POST**

Path: **roast-my-code**

Response Mode: **Response Node**

 *Response Mode must be set to "Response Node" — this tells n8n to wait for the Respond to Webhook node before replying to the browser.*

Step 3 — Add the Basic LLM Chain Node

Click + after the Webhook node → search for Basic LLM Chain → select it.

Set these parameters:

Prompt (User Message): **={{ \$json.body.code }}**

Require Specific Output Format: **OFF (toggle off)**

Enable Fallback Model: **OFF (toggle off)**

Under Chat Messages:

Type Name or ID: **System**

System Message — paste exactly:

You are an experienced senior software developer and code reviewer with 20 years of experience. When someone submits code, you review it professionally and helpfully. Always identify issues and suggest improvements. Format your response EXACTLY as:

 **CODE REVIEW:**
<brief assessment of what the code does and its main issues>

 **ISSUES FOUND:**
1. <issue one>
2. <issue two>

 **IMPROVEMENTS:**
1. <improvement one>
2. <improvement two>

Step 4 — Add the Ollama Model Node

Inside the Basic LLM Chain node, look at the bottom — you will see a Model* slot with a small + button.

Step 4a Click the + on the Model slot — → search for Ollama → select it

Step 4b Set the Credential Base URL — → http://ollama:11434

Step 4c Set the Model — → tinyllama

Step 4d Click Test Connection — → confirm green  message

 *Inside Docker, Ollama is reached by its container service name "ollama" — not "localhost". Always use http://ollama:11434*

Step 5 — Add the Respond to Webhook Node

Click + after the Basic LLM Chain node → search for Respond to Webhook → select it.

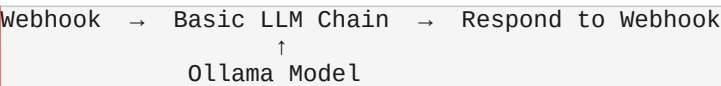
Set these parameters:

Respond With: **JSON**

Response Body: `={{ JSON.stringify({ roast: $json.text }) }}`

 *Copy the Response Body expression exactly as shown including the `={{ }}` wrapper.*

Step 6 — Verify Canvas



Step 7 — Publish and Test

Step 7a Click the Publish button — top right corner of n8n

Step 7b Open code_reviewer_tester.html — in your browser

Step 7c Paste sample code:

```
def divide(a, b):  
    return a / b
```

Step 7d Click  Review My Code — and wait a few seconds

Troubleshooting

 Error / Symptom	 Fix
Webhook returns: not registered for GET requests	Use code_reviewer_tester.html to send a POST request — not the browser address bar.
Server returned 500	Workflow not published, or Ollama container not running. Click Publish.
Server returned 404	Wrong webhook URL. After publishing use /webhook/ not /webhook-test/
Ollama node cannot connect	Base URL must be http://ollama:11434 — not localhost.

Bonus Challenges

 **Change the review style** — Edit the system prompt to review like a strict professor or security expert.

 **Switch the model** — Pull phi3:mini and change the Ollama node model. Compare review quality.



Exercise 2 — Explain Like I'm 5

🕒 15 min

🏠 phi3:mini

📡 3 Nodes

📄 Ex 2

What You Will Build

A 3-node n8n workflow that accepts any CS topic via HTTP POST, sends it to phi3:mini, and returns a simple friendly explanation that anyone can understand — no technical terms, no jargon.

Workflow architecture:

```
Webhook → Basic LLM Chain → Respond to Webhook
          ↑
          Ollama Model (phi3:mini)
```

Key Learning — Prompt Engineering

Prompt Style	Result
No prompt	Long technical explanation with jargon
Simple instruction	Better but still sometimes too long
ELI5 prompt (this lab)	Short, friendly, simple analogies — perfect for beginners

Pre-Requisites

- ☐ Docker Desktop is running
- ☐ n8n is accessible at <http://localhost:5678>
- ☐ Ollama container is running inside Docker
- ☐ phi3:mini model is pulled (see Step 0 below)

Step 0 — Pull the AI Model

```
docker exec -it ollama ollama pull phi3:mini
```

phi3:mini (~2.3 GB) gives the best ELI5 results. Wait until you see "success" before continuing.

Step 1 — Create a New Workflow in n8n

Step 1a Click + New Workflow — top left of the n8n dashboard

Step 1b Name the workflow — "Explain Like Im 5"

Step 1c Click Save

Step 2 — Add the Webhook Node

Click the + button on the canvas → search for Webhook → select it.

HTTP Method: **POST**

Path: `explain-eli5`

Response Mode: `Response Node`

Step 3 — Add the Basic LLM Chain Node

Click + after the Webhook node → search for Basic LLM Chain → select it.

Set Source for Prompt to "Define below" first, then:

Prompt (User Message) — Expression mode: `=Explain {{ $json.body.topic }} like I am a 5 year old child. Use simple words only. Keep it under 5 sentences. Never use technical terms.`

Under Chat Messages:

Type Name or ID: `System`

System Message:

You are a friendly teacher who explains things simply.
Always use easy words. Never use technical terms.
Keep answers under 5 sentences.

 Make sure the Prompt field is in Expression mode — click the fx button on the left of the field.

Step 4 — Add the Ollama Model Node

Step 4a Click the + on the Model* slot — → search for Ollama → select it

Step 4b Set the Credential Base URL — → `http://ollama:11434`

Step 4c Set the Model — → `phi3:mini`

Step 4d Click Test Connection — → confirm green  message

Step 5 — Add the Respond to Webhook Node

Click + after the Basic LLM Chain → search for Respond to Webhook → select it.

Respond With: `JSON`

Response Body: `={{ JSON.stringify({ explanation: $json.text }) }}`

Step 6 — Publish and Test

Step 6a Click Publish — top right corner of n8n

Step 6b Open `eli5_tester.html` — in your browser

Step 6c Select `phi3:mini` from the model buttons

Step 6d Type a topic or click a chip: — e.g. "What is RAM in a computer"

Step 6e Click  Explain It!

Bonus Challenges



Change the audience — Modify the prompt to explain like a grandparent or a journalist.



Compare two models — Select phi3:mini and qwen2.5:3b for the same topic — compare explanations.



Exercise 3 — AI File Summariser

🕒 15 min

🤖 phi3:mini

📡 6 Nodes

📄 Ex 3

What You Will Build

A 6-node n8n workflow that reads a text document from inside a Docker container, sends it to phi3:mini for summarisation, and saves the result as a new file. Students use docker cp commands to upload and download files.

Workflow architecture:

```
Manual Trigger → Read/Write Files from Disk → Extract from File
                                                    ↓
                                                    Basic LLM Chain ← Ollama Model (phi3:mini)
                                                    ↓
                                                    Convert to File → Read/Write Files from Disk
```

Windows Path	Inside n8n Container
shared\document.txt	/home/node/.n8n-files/document.txt
shared\summary.txt	/home/node/.n8n-files/summary.txt

Pre-Requisites

- ☐ Docker Desktop is running
- ☐ n8n is accessible at <http://localhost:5678>
- ☐ phi3:mini model is pulled

Step 0 — Pull the AI Model

```
docker exec -it ollama ollama pull phi3:mini
```

Step 1 — Create n8n Files Folder

```
docker exec -it n8n mkdir -p /home/node/.n8n-files
```

⚠️ This folder is lost every time Docker restarts. Always run this command at the start of each session.

Step 2 — Prepare and Upload Document

Create document.txt in your shared folder, then upload:

```
# Navigate to your folder
cd D:\ai-starter4\self-hosted-ai-starter-kit\shared

# Upload to container
docker cp document.txt n8n:/home/node/.n8n-files/document.txt

# Verify upload
docker exec -it n8n ls /home/node/.n8n-files/
```

Step 3 — Create a New Workflow in n8n

Step 3a Click + New Workflow — name it "AI File Summariser"

Step 3b Click Save

Step 4 — Add the Manual Trigger Node

Click the + button on the canvas → search for Manual Trigger → select it.

No parameters needed — this node starts the workflow when you click Execute Workflow.

Step 5 — Add the Read/Write Files from Disk Node (Read)

Click + after Manual Trigger → search for Read/Write Files from Disk → select it.

Operation: Read File(s) From Disk

File Selector: /home/node/.n8n-files/document.txt

Step 6 — Add the Extract from File Node

Click + after Read/Write Files from Disk → search for Extract from File → select it.

Operation: Extract From Text File

Step 7 — Add the Basic LLM Chain Node

Click + after Extract from File → search for Basic LLM Chain → select it.

Source for Prompt: Define below

Prompt (User Message) — Expression mode: `={{ $json.data }}`

Type Name or ID: System

System Message:

You are a professional document summariser. Summarise the following text in exactly 3 clear sentences. Be concise and clear. Never add extra commentary. Just the summary.

 The Prompt field must be in Expression mode — click the fx button on the left of the field.

Step 8 — Add the Ollama Model Node

Step 8a Click the + on the Model* slot — → search for Ollama → select it

Step 8b Set the Credential Base URL — → http://ollama:11434

Step 8c Set the Model — → phi3:mini

Step 8d Click Test Connection — → confirm green  message

Step 9 — Add the Convert to File Node

Click + after Basic LLM Chain → search for Convert to File → select it.

Operation: Convert to Text File
Text Input Field: text
Put Output File in Field: file
File Name: summary.txt

Step 10 — Add the Write File Node

Click + after Convert to File → search for Read/Write Files from Disk → select it.

Operation: Write File to Disk
File Path: /home/node/.n8n-files/summary.txt
Input Binary Field: file

⚠️ If you see an Append option — remove it by clicking the trash icon. Otherwise the summary will keep adding on every run.

Step 11 — Execute and Download

Step 11a Click Execute Workflow — NOT Publish — this uses Manual Trigger

Step 11b Watch all 6 nodes turn green ✓

Step 11c Download the summary:

```
docker cp n8n:/home/node/.n8n-files/summary.txt .  
type summary.txt
```

Troubleshooting

✗ Error / Symptom	✓ Fix
Access to file is not allowed	File path must be /home/node/.n8n-files/ — n8n only allows access to this folder.
Could not find file in container	Run docker cp to upload document.txt first. Run mkdir if the folder is missing.
Summary is empty or undefined	Prompt is not in Expression mode. Click the fx button and verify the field shows actual text.
.n8n-files folder missing after restart	Run: docker exec -it n8n mkdir -p /home/node/.n8n-files

Bonus Challenges

- 🌐 Change the summary language — Modify the system prompt to return the summary in Arabic or Tamil.
- 🔄 Try a different model — Change the Ollama Model to qwen2.5:3b and compare summary quality.

 Exercise 4 — Simple HTTP Request

 5 min

 Public API

 3 Nodes

 Ex 4

What You Will Build

A 3-node n8n workflow that receives a chat message, calls a public API to fetch a random cat fact, extracts the fact text, and displays it in the chat. This exercise teaches how n8n connects to the internet and retrieves live data from any URL.

Workflow architecture:

```
Chat Trigger → HTTP Request → Edit Fields
              (catfact.ninja)  (extracts fact)
```

Two public APIs you can use:

```
https://catfact.ninja/fact    - returns a random cat fact
https://api.adviceslip.com/advice - returns a random piece of advice
```

Key Learning — HTTP Requests

HTTP Method	What it does
GET	Fetch data from a URL — used in this exercise
POST	Send data to a URL — used in Exercises 1, 2, and 3
JSON response	Most APIs return data in structured key:value format

Pre-Requisites

- ☐ Docker Desktop is running
- ☐ n8n is accessible at <http://localhost:5678>
- ☐ Active internet connection

Step 1 — Create a New Workflow in n8n

Step 1a Click + New Workflow — name it "Simple HTTP Request"

Step 1b Click Save

Step 2 — Add the Chat Trigger Node

Click the + button on the canvas → search for Chat Trigger → select it.

Make Chat Publicly Available: [Toggle ON](#)

 The Chat Trigger opens a built-in chat interface. Every message typed automatically triggers the workflow.

Step 3 — Add the HTTP Request Node

Click + after Chat Trigger → search for HTTP Request → select it.

Method: [GET](#)

URL: <https://catfact.ninja/fact>

The API returns JSON like this:

```
{
  "fact": "Cats have 230 bones in their body.",
  "length": 34
}
```

Step 4 — Add the Edit Fields Node

Click + after HTTP Request → search for Edit Fields → select it.

Click Add Field:

Name: `fact`

Value — Expression mode: `={{ $json.fact }}`

Type: `String`

 The Edit Fields node extracts just the fact field so the chat displays only the text — not the whole JSON response.

Step 5 — Open Chat and Test

Step 5a Click the Chat Trigger node — → click Open Chat

Step 5b Type any message — e.g. "give me a cat fact"

Step 5c See the cat fact in the chat response

Step 6 — Try the Advice API

Change the HTTP Request URL and update Edit Fields:

URL: `https://api.advice Slip.com/advice`

Edit Fields Value: `={{ $json.slip.advice }}`

 catfact.ninja uses \$json.fact (top level) while advice Slip uses \$json.slip.advice (nested). This shows how real APIs differ from each other.

Troubleshooting

 Error / Symptom	 Fix
HTTP Request returns an error	Check internet connection. The URL must be exactly correct.
Edit Fields shows undefined	Field name is wrong. For catfact use \$json.fact. For advice use \$json.slip.advice.
Chat shows full JSON not just the fact	The Edit Fields node is missing or not connected.

Bonus Challenges

 **Try a different API** — Change URL to `https://dog.ceo/api/breeds/image/random`

 **Add AI** — Add a Basic LLM Chain after Edit Fields — ask Ollama to comment on the cat fact.



Exercise 5 — AI Calculator

🕒 10 min

🤖 qwen2.5:3b

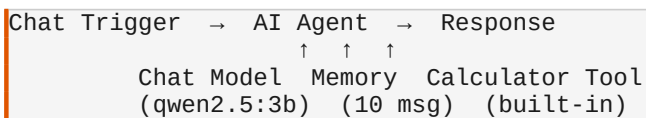
🛠️ 3 + Tools

📄 Ex 5

What You Will Build

A conversational AI calculator using an AI Agent with a built-in Calculator Tool and memory. Unlike a Basic LLM Chain, the AI Agent can call the Calculator Tool to perform exact computations and remember previous questions.

Workflow architecture:



Key Learning — AI Agent vs Basic LLM Chain

Basic LLM Chain (Exercises 1–3)	AI Agent (This Exercise)
Generates text responses only	Can use tools like a calculator
Guesses at maths — may be wrong	Calls Calculator Tool — always correct
No memory between messages	Remembers last 10 messages

Pre-Requisites

- ☐ Docker Desktop is running
- ☐ n8n is accessible at <http://localhost:5678>
- ☐ qwen2.5:3b model is pulled

Step 0 — Pull the AI Model

```
docker exec -it ollama ollama pull qwen2.5:3b
```

Step 1 — Create a New Workflow in n8n

Step 1a Click + New Workflow — name it "AI Calculator"

Step 1b Click Save

Step 2 — Add the Chat Trigger Node

Click + on canvas → search for Chat Trigger → select it.

Make Chat Publicly Available: **Toggle ON**

Step 3 — Add the AI Agent Node

Click + after Chat Trigger → search for AI Agent → select it.

Agent: **Tools Agent**

Under Options:

System Message: **You are a helpful financial assistant, help me the calculator tool**

Step 4 — Add the Ollama Chat Model

Click the + on the Chat Model* slot → search for Ollama Chat Model → select it.

Step 4a Set the Credential Base URL — → http://ollama:11434

Step 4b Set the Model — → qwen2.5:3b

Under Options:

Keep Alive: **1h**

Max Tokens (numPredict): **200**

Step 4c Click Test Connection — → confirm green  message

Step 5 — Add the Simple Memory Node

Click the + on the Memory slot → search for Simple Memory → select it.

Context Window Length: **10**

 Simple Memory remembers the last 10 messages. Students can ask follow-up questions like "now multiply that by 2" and the AI will remember the previous answer.

Step 6 — Add the Calculator Tool

Click the + on the Tool slot → search for Calculator → select it.

No parameters needed — the Calculator Tool works automatically.

Step 7 — Open Chat and Test

Step 7a Click the Chat Trigger node — → click Open Chat

Step 7b Type a maths question:

"What is 128 divided by 4, then multiplied by 3?"

Step 7c Test memory with a follow-up:

"Now add 15% to that result"

Troubleshooting

 Error / Symptom	 Fix
Red warning triangle on AI Agent	Chat Model* slot is not connected. Add the Ollama Chat Model.

✖ Error / Symptom	✔ Fix
AI gives wrong maths answers	The Calculator Tool is not connected to the Tool slot.
AI does not remember previous questions	Simple Memory is not connected to the Memory slot.

Bonus Challenges

-  **Currency conversion** — Ask "convert 1000 USD to OMR at 0.385"
-  **Compound interest** — Ask "calculate compound interest on 10000 at 5% for 5 years"



Exercise 6 — AI Interaction Logger

 15 min

 qwen2.5:3b

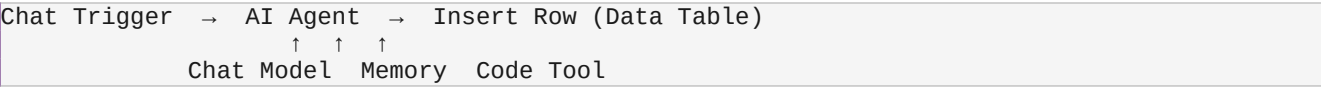
 AI Agent + DB

 Ex 6

What You Will Build

An AI Agent that answers questions AND automatically logs every interaction to an n8n Data Table. The Code Tool analyses the AI response and the Insert Row node saves the question, answer summary, word count, and timestamp permanently.

Workflow architecture:



Step 1 — Create the Data Table in n8n

Before building the workflow, create the Data Table:

- Step 1a** In n8n left sidebar — click the Table icon → + New Table
- Step 1b** Name the table — ai_interaction_log
- Step 1c** Add these columns: — click + Add Column for each

Column Name	Type	Description
question	Text	The question the student asked
answer_summary	Text	First 200 characters of the AI response
word_count	Number	Number of words in the AI response
sentence_count	Number	Number of sentences in the response
reading_time	Number	Estimated reading time in minutes
timestamp	Text	Date and time of the interaction

Step 2 — Create a New Workflow

- Step 2a** Click + New Workflow — name it "AI Interaction Logger"
- Step 2b** Click Save

Step 3 — Add the Chat Trigger Node

Click + on canvas → search for Chat Trigger → select it.

Make Chat Publicly Available: **Toggle ON**

Step 4 — Add the AI Agent Node

Click + after Chat Trigger → search for AI Agent → select it.

Agent: **Tools Agent**

Under Options:

System Message: You are a helpful CS tutor. Answer questions clearly and concisely. Always be educational.

Step 5 — Add the Ollama Chat Model

Click the + on the Chat Model* slot → search for Ollama Chat Model → select it.

Step 5a Set the Credential Base URL — → http://ollama:11434

Step 5b Set the Model — → qwen2.5:3b

Step 5c Click Test Connection — → confirm green  message

Step 6 — Add the Simple Memory Node

Click the + on the Memory slot → search for Simple Memory → select it.

Context Window Length: **10**

Step 7 — Add the Code Tool Node

Click the + on the Tool slot → search for Code Tool → select it.

Language: **JavaScript**

Description: **Analyses text and returns word count, sentence count and reading time**

Paste this code exactly in the JavaScript field:

```
const input = $input.first().json;
const text = input.code || input.query || input._query || '';
const words = text.trim().split(/\s+/).filter(w => w).length;
const sentences = text.split(/[.!?]+/).filter(s => s.trim()).length;
return `Analysed: ${words} words, ${sentences} sentences.`;
```

Step 8 — Add the Insert Row Node

Click + after the AI Agent main output → search for n8n → select Insert Row.

Table: ai_interaction_log

Under Fields to Send — click Add Field for each:

```
question:={{ $('When chat message received').item.json.chatInput }}
answer_summary:={{ $json.output.slice(0, 200) }}
word_count:={{ $json.output.trim().split(/\s+/).length }}
sentence_count:={{ $json.output.split(/[!?]+/).filter(s => s.trim()).length }}
reading_time:={{ Math.ceil($json.output.trim().split(/\s+/).length / 200) }}
timestamp:={{ new Date().toISOString().replace("T", " ").slice(0,19) }}
```

 The question field uses \$('When chat message received').item.json.chatInput to get what the student typed. The word_count and sentence_count are calculated directly from \$json.output.

Step 9 — Test the Workflow

Step 9a Click the Chat Trigger node — → click Open Chat

Step 9b Ask a question:

"What is a binary search tree?"

Step 9c Go to Tables in n8n sidebar — → open ai_interaction_log

Step 9d See your question saved as a new row 

Troubleshooting

 Error / Symptom	 Fix
Insert Row says table not found	Create ai_interaction_log in Step 1 — name must match exactly.
question field is empty in table	Use \$('When chat message received').item.json.chatInput — not \$json.chatInput.
word_count shows 0	Remove the Code Tool — word_count is now calculated in the Insert Row expression directly.

Bonus Challenges

 **View statistics** — After 10 interactions sort by word_count — see which question got the longest answer.

 **Export the data** — In the Data Table view look for Export — download all interactions as a CSV file.



Exercise 7 — RAG Pipeline

 20 min

 mxbai + qwen2.5:3b

 Two Workflows

 Ex 7

What You Will Build

Two connected n8n workflows that create a question-answering system from your own PDF. Workflow 1 ingests the PDF into Qdrant. Workflow 2 uses an AI Agent to answer questions by retrieving relevant content from the database.

Workflow 1 — Data Ingestion (run once):

```
n8n Form (upload PDF) → Qdrant Vector Store (insert)
                        ↑
                Default Data Loader  Embeddings Ollama (mxbai-embed-large)
                        ↑
                Recursive Text Splitter (chunk: 200, overlap: 50)
```

Workflow 2 — RAG Chatbot (run every time you ask a question):

```
Chat Trigger → AI Agent → Response
              ↑   ↑   ↑
        Chat Model Memory Qdrant Retriever Tool
        (qwen2.5:3b) (10) ↑
                   Embeddings Ollama1 (mxbai-embed-large)
```

Key Learning — What is RAG?

1. Ingest — PDF is split into small chunks of text
2. Embed — Each chunk is converted into a numeric vector
3. Store — Vectors are saved in Qdrant vector database
4. Retrieve — Question is embedded and matched to similar chunks
5. Generate — AI answers using the retrieved chunks as context

Pre-Requisites

- ☐ Docker Desktop is running
- ☐ n8n is accessible at <http://localhost:5678>
- ☐ Qdrant accessible at <http://localhost:6333/dashboard>
- ☐ mxbai-embed-large pulled
- ☐ qwen2.5:3b pulled
- ☐ What-is-RAG-sourced.pdf ready

Step 0 — Pull the Required Models

```
docker exec -it ollama ollama pull mxbai-embed-large
docker exec -it ollama ollama pull qwen2.5:3b
```

Step 1 — Create the Qdrant Collection

Step 1a Open Qdrant dashboard — <http://localhost:6333/dashboard>

Step 1b Click Create Collection

Collection Name: `rag_collection`

Vector Size: `1024`

Distance: `Cosine`

Step 1c Click Create

 Vector Size 1024 matches the output size of mxbai-embed-large. This must match exactly.

Workflow 1 — Data Ingestion

Step 2 — Create Workflow 1

Step 2a Click + New Workflow — name it "RAG — Data Ingestion"

Step 2b Click Save

Step 3 — Add the n8n Form Trigger Node

Click + on canvas → search for n8n Form Trigger → select it.

Form Title: `Add your file here`

Under Form Fields — click Add Field:

Field Label: `File`

Field Type: `File`

Accept File Types: `.pdf`

Required: `Toggle ON`

Step 4 — Add the Qdrant Vector Store Node (Insert)

Click + after Form Trigger → search for Qdrant Vector Store → select it.

Operation Mode: `Insert Documents`

Collection: `rag_collection`

Step 5 — Add Embeddings Ollama (Ingestion)

Click the + on the Embeddings slot of the Qdrant Vector Store node → search for Embeddings Ollama → select it.

Base URL: `http://ollama:11434`

Model: `mxbai-embed-large:latest`

Step 6 — Add the Default Data Loader Node

Click the + on the Document slot → search for Default Data Loader → select it.

Data Type: `Binary`

Step 7 — Add the Recursive Character Text Splitter

Inside the Default Data Loader, click the + on the Text Splitter slot → search for Recursive Character Text Splitter → select it.

Chunk Size: `200`

Chunk Overlap: `50`

Step 8 — Ingest the PDF

Step 8a Click **Form Trigger** node — → copy the Form URL

Step 8b Open the Form URL in browser

Step 8c Select **What-is-RAG-sourced.pdf** — → click Submit

Step 8d Watch all nodes turn green 

 After ingestion go to <http://localhost:6333/dashboard> → [rag_collection](#) → you should see 18 points stored.

Workflow 2 — RAG Chatbot

Step 9 — Create Workflow 2

Step 9a Click **+ New Workflow** — name it "RAG — Chatbot"

Step 9b Click **Save**

Step 10 — Add the Chat Trigger Node

Click **+** on canvas → search for **Chat Trigger** → select it.

Make Chat Publicly Available: **Toggle ON**

Initial Messages: **Hi there!** 🙌 **My name is Ramkumar. How can I assist you today?**

Step 11 — Add the AI Agent Node

Click **+** after **Chat Trigger** → search for **AI Agent** → select it.

Agent: **Tools Agent**

Under Options:

System Message: You are a helpful assistant. You MUST use the retriever tool to answer every question. Always call the retriever tool first. Base your answer only on what the retriever returns.

Step 12 — Add the Ollama Chat Model

Click the **+** on the **Chat Model*** slot → search for **Ollama Chat Model** → select it.

Step 12a Set the **Credential Base URL** — → <http://ollama:11434>

Step 12b Set the **Model** — → [qwen2.5:3b](#)

Under Options:

Keep Alive: **1h**

Max Tokens (numPredict): **150**

Step 12c Click **Test Connection** — → confirm green  message

Step 13 — Add the Simple Memory Node

Click the + on the Memory slot → search for Simple Memory → select it.
No parameters needed — default settings work fine.

Step 14 — Add the Qdrant Vector Store Node (Retrieve)

Click the + on the Tool slot → search for Qdrant Vector Store → select it.

Operation Mode: Retrieve Documents (As Tool for AI Agent)
Tool Name: retriever
Tool Description: Retrieve data from a semantic database to answer questions
Qdrant Collection: rag_collection
Limit: 4
Include Metadata: Toggle ON

Scroll down — find the Embedding* slot at the bottom → click + → search Embeddings Ollama → select it:
Base URL: http://ollama:11434
Model: mxbai-embed-large:latest

 This Embeddings Ollama is SEPARATE from the one in Workflow 1. Each Qdrant node needs its own dedicated Embeddings node — they cannot share one.

Step 15 — Test with Sample Questions

Step 15a Click Chat Trigger node — → click Open Chat

Step 15b Try these questions from the PDF:

"What is RAG?"
"What are the five steps of how RAG works?"
"What is the open-book exam analogy?"
"What are the four key components of a RAG system?"
"What is Qdrant used for in RAG?"

 Answers should reference content from the PDF — not general AI knowledge. You should see specific details from the uploaded document.

Troubleshooting

 Error / Symptom	 Fix
Qdrant collection not found	Create rag_collection first in Step 1 — name must be exactly "rag_collection".
Vector dimension mismatch	Vector Size must be 1024 in Qdrant — matches mxbai-embed-large output.
AI says "I could not find this"	The Embeddings Ollama1 is missing from the Qdrant retriever Embedding* slot.
Tool Name is missing	Set Tool Name to "retriever" in the Qdrant Vector Store1 parameters.
AI gives generic answers not from PDF	Qdrant retriever not connected as Tool. Add it to the Tool slot of the AI Agent.

Bonus Challenges



Upload a different PDF — Try ingesting a chapter from your textbook. Ask questions about it.



Change the language — Modify the System Message to respond in Arabic.